SIMATS ENGINEERING

ASSESSMENT TOOL 2 - DESIGN TASK

Course Name: INTERNET PROGRAMMING Course Code: CSA43

CO-2: Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Weightage: 20% Total Marks: 20

Name:	Soniah Naren
Register No:	192311012
Department / Section:	CSE Core

Code of Conduct

I Soniah Naren / 192311012 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Contents

- Q1 Responsive Navigation Bar
- Q2 Product Hover Effect
- Q3 Layout Selection (Flexbox vs Grid)
- Q4 Mobile-First Blog Layout
- Q5 JavaScript Event Handling for Dynamic Menu

Tools used: HTML5, CSS3, JavaScript (ES6), Google Chrome. Screenshots were captured from the executed code at different screen widths.

Question 1: Responsive Navigation Bar Design (4 marks)

(a) Key components of an e-commerce navigation bar

- **Logo / brand name** - links back to the home page.
- **Main links** - Home, Shop, Categories, Deals, Contact.
- **Search box** - lets users find products quickly.
- **Cart** with item count and **Login / Account** link.
- **Hamburger icon** - shown only on small screens to open the menu.

(b) Procedure to design the responsive navbar

- Create `q1_navbar.html` and add the viewport meta tag so the page scales on phones.
- Write a <nav> containing the logo, a <ul> of links, a search box, the cart link and a hamburger label.
- Use **Flexbox** (`display:flex; justify-content:space-between; align-items:center`) to place items in one row on desktop.
- Add a media query at **992px** (tablet) to reduce the search width and gaps.
- Add a media query at **768px** (mobile): hide the links and search, show the hamburger, and make the menu a vertical dropdown.
- Use a hidden checkbox + `:checked ~ .menu` so the hamburger works with pure CSS (no JavaScript).
- Open the file in Chrome and resize the window to check each layout.

Code (q1_navbar.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Q1 - Responsive Navbar</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }
  body { font-family: Arial, sans-serif; }
  .navbar { display: flex; align-items: center; justify-content: space-between;
            background: #1f2937; color: #fff; padding: 12px 20px; position: relative; }
  .logo { font-size: 22px; font-weight: bold; color: #fbbf24; text-decoration: none; }
  .menu { display: flex; list-style: none; gap: 18px; }
  .menu a { color: #fff; text-decoration: none; padding: 6px 4px; }
  .menu a:hover { color: #fbbf24; }
  .search { padding: 7px 10px; border: none; border-radius: 4px; width: 180px; }
  .actions { display: flex; align-items: center; gap: 14px; }
  .cart { background: #fbbf24; color: #111; padding: 6px 12px; border-radius: 4px;
          font-weight: bold; text-decoration: none; }
  .toggle-box { display: none; }              /* hidden checkbox used for the hamburger */
  .hamburger { display: none; font-size: 28px; cursor: pointer; }

  /* Tablet: shrink search box and spacing */
  @media (max-width: 992px) {
    .search { width: 120px; }
    .menu { gap: 10px; }
  }
  /* Mobile: collapse links into a vertical dropdown */
  @media (max-width: 768px) {
    .hamburger { display: block; }
    .search, .actions .login { display: none; }
    .menu { display: none; flex-direction: column; position: absolute; top: 100%; left: 0;
            width: 100%; background: #374151; padding: 10px 20px; gap: 0; z-index: 10; }
    .menu li a { display: block; padding: 12px 0; border-bottom: 1px solid #4b5563; }
    .toggle-box:checked ~ .menu { display: flex; }
  }
</style>
</head>
<body>
<nav class="navbar">
  <a href="#" class="logo">ShopEasy</a>
  <input type="checkbox" id="toggle" class="toggle-box">
  <ul class="menu">
    <li><a href="#">Home</a></li>
    <li><a href="#">Shop</a></li>
```

```
    <li><a href="#">Categories</a></li>
    <li><a href="#">Deals</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
  <div class="actions">
    <input class="search" type="text" placeholder="Search products...">
    <a href="#" class="login" style="color:#fff">Login</a>
    <a href="#" class="cart">Cart (2)</a>
    <label for="toggle" class="hamburger">&#9776;</label>
  </div>
</nav>
<p style="padding:20px">Page content goes here...</p>
</body>
</html>
```

(c) Output - how the navigation adapts

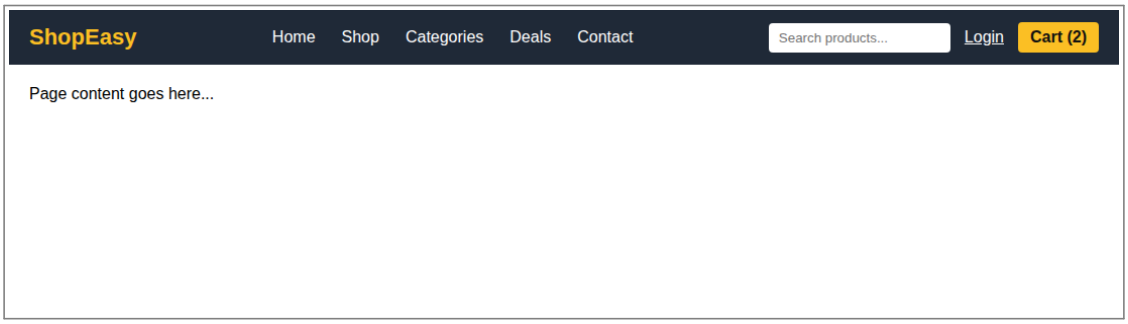


Fig 1.1 Desktop (1100px): full menu, search box, login and cart in one row

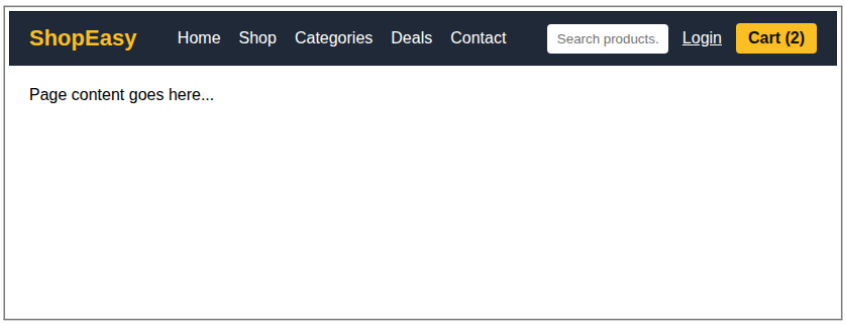


Fig 1.2 Tablet (820px): search box and spacing are reduced



Fig 1.3 Mobile (390px): hamburger icon, menu hidden

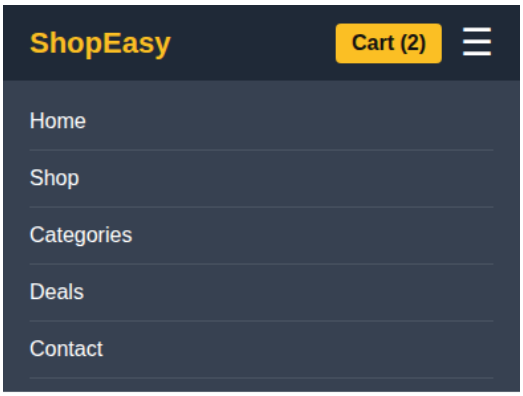


Fig 1.4 Mobile: menu opened after tapping the hamburger

(d) Evaluation and improvements

- **Usability:** the most important actions (search, cart) stay visible on every screen size; the dropdown links are large enough to tap.

- **Improvements:** make the menu sticky (`position:sticky`), add `aria-label` and keyboard support to the hamburger, show a cart item badge, highlight the active page, and add mobile search as an icon that expands.

Question 2: Product Hover Effect (4 marks)

(a) Suitable CSS properties for hover effects

- `:hover` pseudo-class to detect the mouse over the card.
- `transform` (translate, scale) - moves or zooms an element.
- `box-shadow` - gives a lifted (3D) look.
- `opacity`, `background-color`, `filter` - fade or colour changes.

(b) Suitable animation / transition

A CSS **transition** is best for hover because it simply animates between two states. The card uses `transition: transform 0.3s ease, box-shadow 0.3s ease`. No keyframe animation is needed, which keeps the code simple.

(c) Procedure to design the hover card

- Create `q2_hover.html` with three product cards (image area, name, price and an "Add to Cart" bar).
- Give each card a base style: width, white background, border-radius and a light shadow.
- Add the transition property on the card so changes happen smoothly.
- On `.card:hover` move the card up by 10px (`translateY(-10px)`) and increase the shadow.
- On hover also zoom the image (`scale(1.12)`) inside an `overflow:hidden` wrapper, and slide up the hidden "Add to Cart" bar by changing opacity and transform.
- Run the page, move the mouse over a card and capture the screenshot.

Code (q2_hover.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Q2 - Product Hover Effect</title>
<style>
  body { font-family: Arial, sans-serif; background: #f3f4f6; padding: 30px; }
  .grid { display: flex; gap: 25px; flex-wrap: wrap; justify-content: center; }
  .card { width: 230px; background: #fff; border-radius: 10px; overflow: hidden;
          box-shadow: 0 2px 6px rgba(0,0,0,0.15); position: relative;
          transition: transform 0.3s ease, box-shadow 0.3s ease; }
  .card:hover { transform: translateY(-10px); box-shadow: 0 14px 24px rgba(0,0,0,0.25); }
  .img-wrap { height: 170px; overflow: hidden; }
  .img { height: 100%; transition: transform 0.4s ease; }
  .card:hover .img { transform: scale(1.12); }
  .info { padding: 14px 14px 56px; }   /* room for the slide-up button */
  .price { color: #dc2626; font-weight: bold; }
  .btn { position: absolute; left: 0; right: 0; bottom: 0; background: #111827; color: #fff;
         text-align: center; padding: 12px; opacity: 0; transform: translateY(100%);
         transition: opacity 0.3s ease, transform 0.3s ease; }
  .card:hover .btn { opacity: 1; transform: translateY(0); }
  @media (prefers-reduced-motion: reduce) { .card, .img, .btn { transition: none; } }
</style>
</head>
<body>
<h2 style="text-align:center;margin-bottom:25px">Featured Products</h2>
<div class="grid">
  <div class="card"><div class="img-wrap"><div class="img" style="background:linear-
      gradient(135deg,#60a5fa,#1d4ed8)"></div></div>
    <div class="info"><h3>Wireless Headphones</h3><p class="price">Rs. 2,499</p></div><div class="btn">Add to
       Cart</div></div>
  <div class="card"><div class="img-wrap"><div class="img" style="background:linear-
      gradient(135deg,#f9a8d4,#be185d)"></div></div>
    <div class="info"><h3>Smart Watch</h3><p class="price">Rs. 3,999</p></div><div class="btn">Add to Cart</div></div>
  <div class="card"><div class="img-wrap"><div class="img" style="background:linear-
      gradient(135deg,#86efac,#15803d)"></div></div>
    <div class="info"><h3>Bluetooth Speaker</h3><p class="price">Rs. 1,799</p></div><div class="btn">Add to
       Cart</div></div>
</div>
</body>
</html>
```

Output

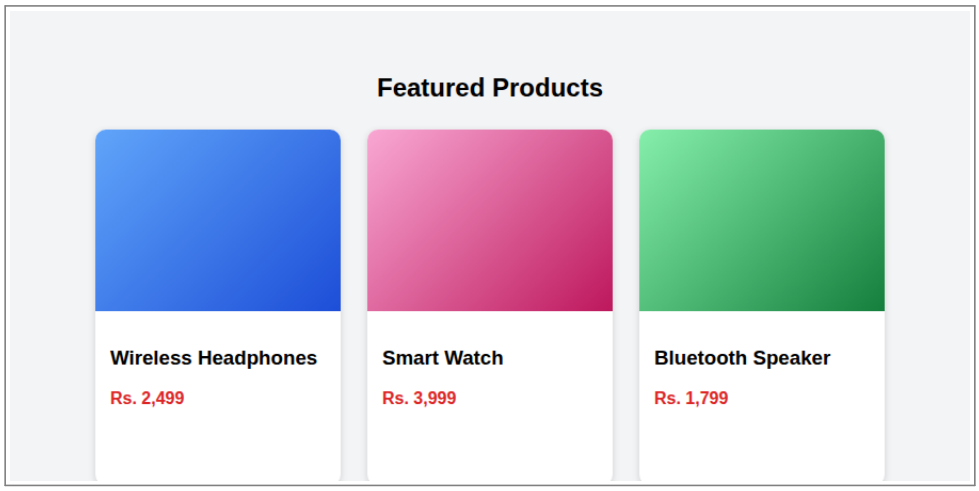


Fig 2.1 Normal state of the product cards

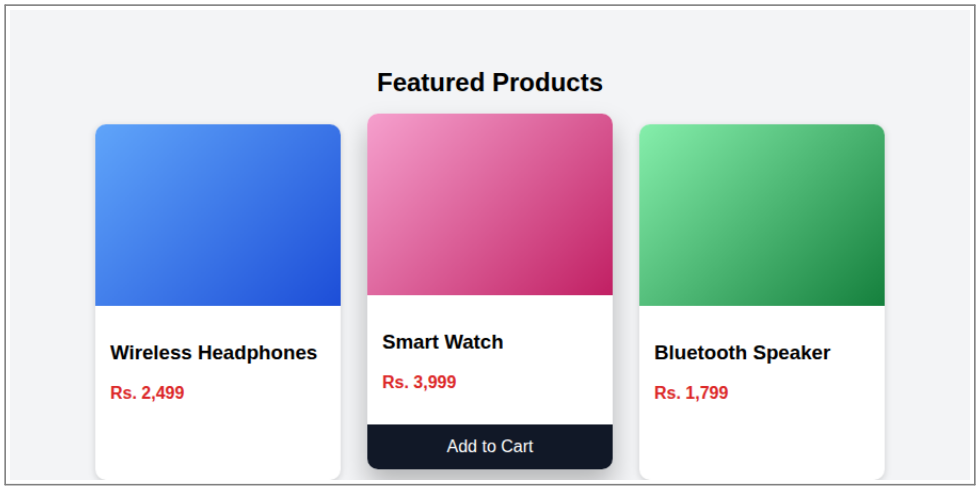


Fig 2.2 Hover on "Smart Watch": card lifts, image zooms, shadow grows, Add to Cart bar appears

(d) Performance and user experience

- **Performance:** only `transform` and `opacity` are animated. These are handled by the GPU and do not force the browser to recalculate layout, so the animation stays smooth. Animating width, height or margin is avoided.
- **UX:** short duration (0.3s) gives instant feedback without feeling slow; the "Add to Cart" button appears only when needed so the card stays clean.
- **Accessibility / mobile:** touch screens have no hover, so the button should also be visible on tap or always shown on small screens; `prefers-reduced-motion` is used to switch off animation for users who need it.

Question 3: Layout Selection - Flexbox vs Grid (4 marks)

(a) Layout requirements of a dashboard

- A fixed **header** row, a **sidebar** column, a flexible **main** area and a **footer** row (rows and columns together).
- Main area has equal-sized **stat cards** and one full-width chart area.
- Items must be aligned (header text left/right, footer text centred) and the layout must collapse to one column on mobile.

(b) Flexbox vs Grid

Feature	Flexbox	CSS Grid
Dimension	One-dimensional (row OR column)	Two-dimensional (rows AND columns)
Best for	Navbars, aligning items, small components	Full page layouts, dashboards, card grids
Placement	Items flow in order	Items placed in exact cells / named areas
Responsive cards	Needs wrapping and width tricks	`repeat(auto-fit, minmax())` in one line
Alignment	Excellent (justify / align)	Excellent (works on both axes)

(c) Choice and justification

CSS Grid is chosen for the overall dashboard because it has rows and columns at the same time, and `grid-template-areas` lets the layout be read like a map (header, sidebar, main, footer). **Flexbox** is still used inside the header and footer to align their content. Using Grid for the page and Flexbox for components is the usual best practice.

Procedure

- Create `q3_dashboard.html` with header, aside, main and footer elements.
- Set `.dashboard {display:grid}` with columns `200px 1fr`, rows `60px 1fr 50px` and `grid-template-areas`.
- Assign each element a `grid-area` name.
- Make the main area its own grid with `repeat(auto-fit, minmax(160px,1fr))` so the cards resize automatically; the chart card spans all columns with `grid-column: 1 / -1`.
- Use Flexbox in the header and footer for alignment.
- Add a media query (max-width 700px) that redefines the areas to a single column.

Code (q3_dashboard.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Q3 - Dashboard Layout (CSS Grid + Flexbox)</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }
  body { font-family: Arial, sans-serif; background: #e5e7eb; }
  /* CSS Grid: 2-D page structure */
  .dashboard { display: grid; min-height: 100vh; gap: 12px; padding: 12px;
               grid-template-columns: 200px 1fr;
               grid-template-rows: 60px 1fr 50px;
               grid-template-areas: "header header"
                                    "sidebar main"
                                    "footer  footer"; }
  .header  { grid-area: header;  background: #1e3a8a; color: #fff; padding: 0 20px;
             display: flex; align-items: center; justify-content: space-between; } /* Flexbox: 1-D alignment */
  .sidebar { grid-area: sidebar; background: #fff; padding: 15px; }
  .sidebar li { list-style: none; padding: 10px 0; border-bottom: 1px solid #ddd; }
  .main    { grid-area: main; display: grid; gap: 12px;
             grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)); align-content: start; }
  .card    { background: #fff; padding: 20px; border-radius: 8px; text-align: center; }
  .card b  { display: block; font-size: 26px; color: #1e3a8a; margin-top: 6px; }
```

```
  .wide    { grid-column: 1 / -1; height: 140px; }
  .footer  { grid-area: footer; background: #1f2937; color: #fff; display: flex;
             align-items: center; justify-content: center; }
  @media (max-width: 700px) {
    .dashboard { grid-template-columns: 1fr; grid-template-rows: auto;
                 grid-template-areas: "header" "sidebar" "main" "footer"; }
  }
</style>
</head>
<body>
<div class="dashboard">
  <header class="header"><h3>Admin Dashboard</h3><span>Welcome, Admin</span></header>
  <aside
      class="sidebar"><ul><li>Overview</li><li>Orders</li><li>Customers</li><li>Reports</li><li>Settings</li></ul></as
      ide>
  <main class="main">
    <div class="card">Users<b>1,250</b></div>
    <div class="card">Orders<b>340</b></div>
    <div class="card">Revenue<b>Rs. 85K</b></div>
    <div class="card">Visitors<b>5,420</b></div>
    <div class="card wide">Sales Chart Area</div>
  </main>
  <footer class="footer">&copy; 2026 Dashboard</footer>
</div>
</body>
</html>
```

Output

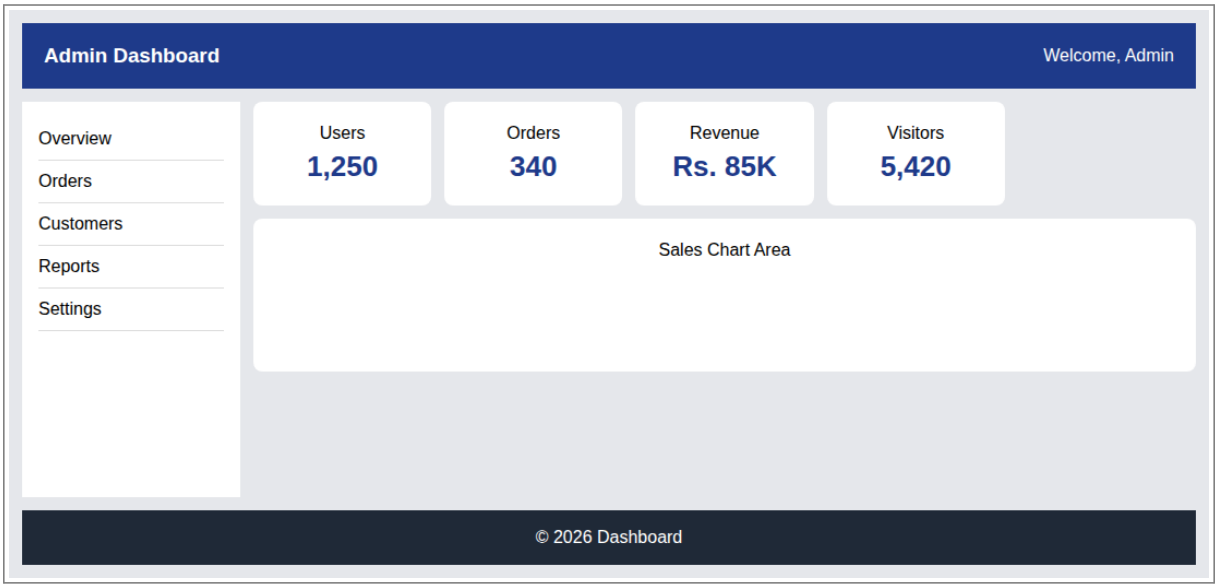


Fig 3.1 Desktop (1100px): sidebar + main area with four stat cards and a full-width chart box

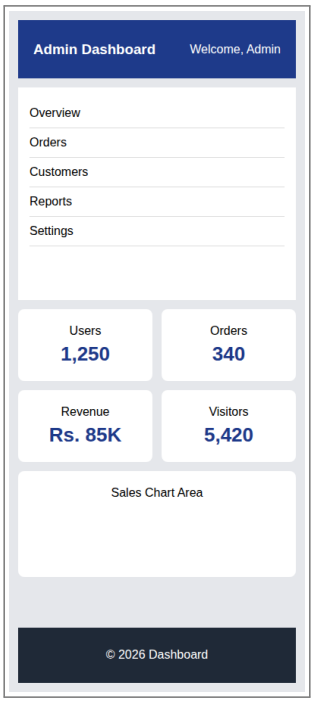


Fig 3.2 Mobile (390px): all areas stack in one column

(d) Scalability and responsiveness

- Adding more cards needs no CSS change: `auto-fit` places them into new rows automatically.
- Named grid areas make the layout easy to rearrange for each breakpoint with just one media query.
- The 1fr unit and minmax() let the main area fill the remaining space on any screen size, so the design scales from phones to large monitors.

Question 4: Mobile-First Blog Layout (4 marks)

(a) Key sections of a blog layout

- **Header** with blog title and navigation links.
- **Main article** - featured image, heading, post date/author and text.
- **Sidebar** - categories, recent posts or search.
- **Footer** - copyright and links.

(b) Procedure for the mobile-first design

- Create `q4_blog.html` with header, a container holding article and aside, and a footer; add the viewport meta tag.
- Write the **base CSS for mobile first**: a single-column layout using `display:grid` with a gap, full-width hero image and 18px text.
- Limit paragraph width with `max-width:65ch` and use `line-height:1.6` for readability.
- Use `min-width` media queries to enhance for larger screens: at **768px** change to two columns (`2fr 1fr`) and at **1024px** use `3fr 1fr`, a larger font and taller image.
- Test in Chrome by resizing to 390px, 800px and 1200px.

Code (q4_blog.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Q4 - Mobile-First Blog</title>
<style>
  /* ---------- BASE = MOBILE (single column) ---------- */
  * { box-sizing: border-box; margin: 0; padding: 0; }
  body { font-family: Georgia, serif; font-size: 18px; line-height: 1.6; color: #222; background: #fafafa; }
  header { background: #0f766e; color: #fff; padding: 16px; }
  nav a { color: #fff; margin-right: 12px; font-family: Arial, sans-serif; font-size: 16px; }
  .container { display: grid; gap: 16px; padding: 16px; max-width: 1100px; margin: auto; }
  article, aside { background: #fff; padding: 16px; border-radius: 6px; }
  article h2 { margin-bottom: 8px; }
  article p  { max-width: 65ch; margin-bottom: 12px; }
  .meta { font-size: 14px; color: #555; margin-bottom: 10px; }
  img.hero { width: 100%; height: 140px; background: linear-gradient(135deg,#5eead4,#0f766e); border-radius: 6px;
      margin-bottom: 10px; display:block; }
  footer { background: #134e4a; color: #fff; text-align: center; padding: 14px; font-size: 14px; }
  a:focus { outline: 3px solid #fbbf24; }

  /* ---------- TABLET: 768px and up ---------- */
  @media (min-width: 768px) {
    body { font-size: 19px; }
    .container { grid-template-columns: 2fr 1fr; padding: 24px; }   /* article + sidebar */
    img.hero { height: 220px; }
  }
  /* ---------- DESKTOP: 1024px and up ---------- */
  @media (min-width: 1024px) {
    body { font-size: 20px; }
    .container { grid-template-columns: 3fr 1fr; gap: 28px; }
    img.hero { height: 280px; }
  }
</style>
</head>
<body>
<header>
  <h1>Travel Diaries</h1>
  <nav><a href="#">Home</a><a href="#">Posts</a><a href="#">About</a><a href="#">Contact</a></nav>
</header>
<div class="container">
  <article>
    <img class="hero" alt="Sunrise over the hills"
        src="">
    <h2>A Weekend in Kodaikanal</h2>
    <p class="meta">Posted on 27 Oct 2026 by Admin</p>
    <p>Kodaikanal is a calm hill town known for its misty mornings, lakes and pine forests. We started early to catch
        the sunrise from Coaker's Walk.</p>
    <p>The best way to explore is on foot. Carry a light jacket, because the weather changes quickly during the
        day.</p>
```

```
  </article>
  <aside>
    <h3>Categories</h3>
    <p>Destinations</p><p>Food</p><p>Travel Tips</p>
  </aside>
</div>
<footer>&copy; 2026 Travel Diaries</footer>
</body>
</html>
```

(c) Output - how the layout scales

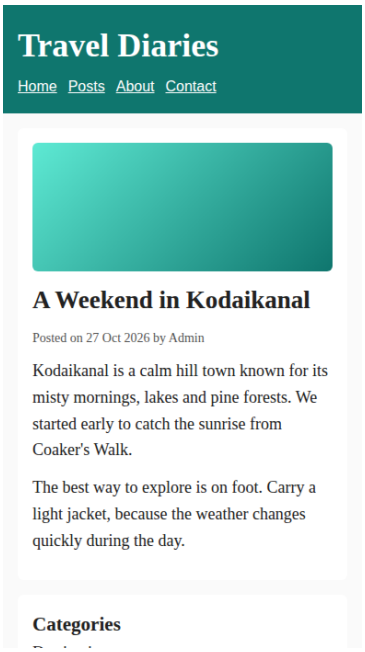


Fig 4.1 Mobile (390px): single column, sidebar below the article

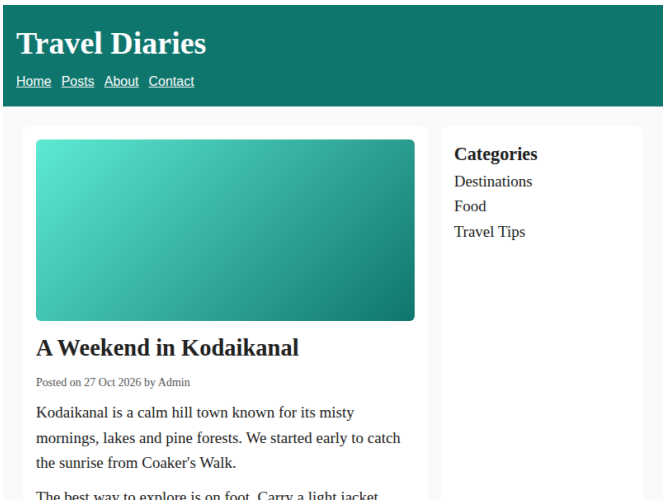


Fig 4.2 Tablet (800px): article and sidebar side by side

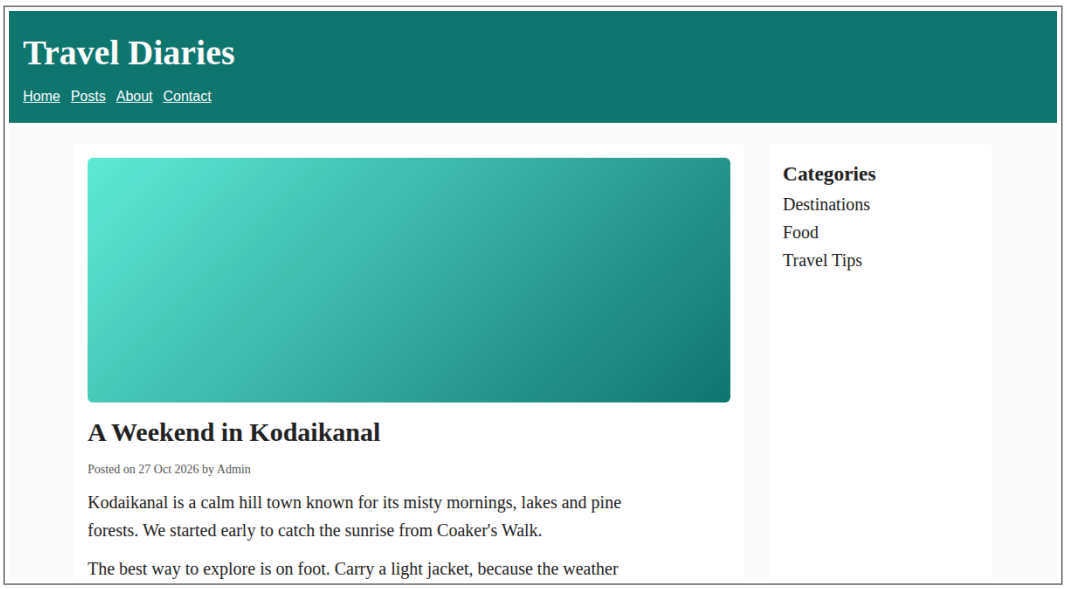


Fig 4.3 Desktop (1200px): wider layout with larger text and image

(d) Accessibility and readability

- **Readability:** 18-20px serif font, 1.6 line height, about 65 characters per line and dark text on a light background (high contrast).
- **Accessibility:** semantic tags (header, nav, article, aside, footer), `alt` text for the image, visible keyboard focus outline and relative units so users can zoom.
- **Improvements:** add a skip-to-content link, use `rem` units for fonts, and check colour contrast ratio is at least 4.5:1.

Question 5: JavaScript Event Handling for Dynamic Menu (4 marks)

(a) Appropriate JavaScript events

Event	Used for
click	Toggle (open / close) a dropdown on button press; select a menu link; works on touch screens
mouseenter / mouseleave	Open the dropdown when the mouse is over a menu item and close when it leaves (desktop hover)
keydown	Close the menu when the Esc key is pressed (keyboard accessibility)
click on document	Close the menu when the user clicks anywhere outside

(b) Event handling logic for the toggle (procedure)

- Create `q5_menu.html` with a nav bar of three items; each has a button and a hidden dropdown (`display:none`).
- CSS rule `.item.open .dropdown {display:block}` shows the dropdown when the class *open* is present.
- In JavaScript, select all items and write `closeAll()` to remove the open class and set `aria-expanded="false"`.
- Write `toggle(item)`: close all others, then open the clicked item if it was not already open (so only one dropdown is open at a time).
- Add **one** click listener on the parent menu (event delegation) to handle all buttons and links.
- Add hover listeners (mouseenter/mouseleave) only when the device supports hover, plus Esc key and outside-click listeners.

(c) Code with implementation of click / hover (q5_menu.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Q5 - Dynamic Menu using JavaScript Events</title>
<style>
  body { font-family: Arial, sans-serif; margin: 0; }
  .bar { background: #312e81; color: #fff; padding: 12px 20px; display: flex; gap: 20px; align-items: center; }
  .item { position: relative; }
  .item > button { background: none; border: none; color: #fff; font-size: 16px; cursor: pointer; padding: 8px 12px; }
  .item > button:hover, .item > button:focus { background: #4338ca; outline: none; }
  .dropdown { display: none; position: absolute; top: 100%; left: 0; min-width: 170px;
              background: #fff; box-shadow: 0 6px 14px rgba(0,0,0,0.25); z-index: 5; }
  .dropdown a { display: block; padding: 10px 14px; color: #111; text-decoration: none; }
  .dropdown a:hover { background: #e0e7ff; }
  .item.open .dropdown { display: block; }
  #msg { padding: 20px; font-size: 18px; }
</style>
</head>
<body>
<nav class="bar" id="menu">
  <strong>MyApp</strong>
  <div class="item"><button aria-expanded="false">Products &#9662;</button>
    <div class="dropdown"><a href="#">Laptops</a><a href="#">Phones</a><a href="#">Tablets</a></div></div>
  <div class="item"><button aria-expanded="false">Services &#9662;</button>
    <div class="dropdown"><a href="#">Repair</a><a href="#">Support</a></div></div>
  <div class="item"><button aria-expanded="false">Account &#9662;</button>
    <div class="dropdown"><a href="#">Profile</a><a href="#">Logout</a></div></div>
</nav>
<p id="msg">Click a menu button, hover over it, or press Esc.</p>

<script>
  const menu = document.getElementById('menu');
  const items = document.querySelectorAll('.item');
  const msg = document.getElementById('msg');

  function closeAll() {
    items.forEach(i => { i.classList.remove('open', 'pinned'); i.querySelector('button').setAttribute('aria-expanded',
        'false'); });
```

```
  }
  function toggle(item) {
    const wasPinned = item.classList.contains('pinned');
    closeAll();                                          // only one dropdown open at a time
    if (!wasPinned) { item.classList.add('open', 'pinned'); item.querySelector('button').setAttribute('aria-expanded',
        'true'); }
  }

  // CLICK event (event delegation: one listener on the parent menu)
  menu.addEventListener('click', e => {
    const btn = e.target.closest('button');
    if (btn) { toggle(btn.parentElement); msg.textContent = 'Clicked: ' + btn.textContent.trim(); }
    const link = e.target.closest('.dropdown a');
    if (link) { e.preventDefault(); msg.textContent = 'You selected: ' + link.textContent; closeAll(); }
  });

  // HOVER events (only on devices that support hover)
  if (window.matchMedia('(hover: hover)').matches) {
    items.forEach(item => {
      item.addEventListener('mouseenter', () => { closeAll(); item.classList.add('open'); });   // hover opens, click
          pins
      item.addEventListener('mouseleave', () => { if (!item.classList.contains('pinned'))
          item.classList.remove('open'); });
    });
  }

  // KEYDOWN event: Esc closes menu
  document.addEventListener('keydown', e => { if (e.key === 'Escape') closeAll(); });

  // Click outside closes menu
  document.addEventListener('click', e => { if (!menu.contains(e.target)) closeAll(); });
</script>
</body>
</html>
```

Output

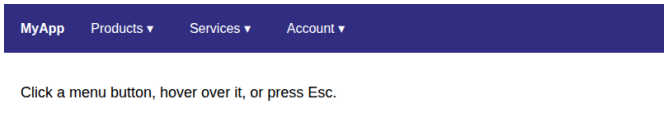


Fig 5.1 Initial state - all dropdowns hidden

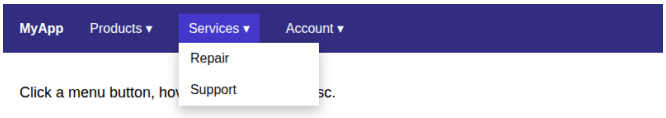


Fig 5.2 Hover over "Account" opens its dropdown

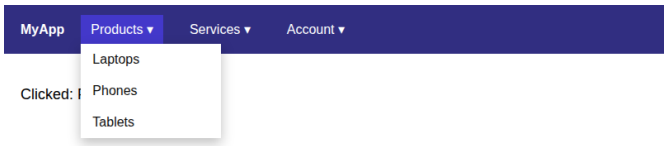


Fig 5.3 Click on "Products" opens and keeps it open

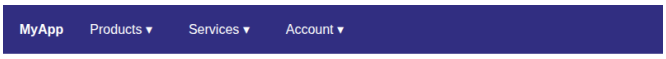


Fig 5.4 Click on "Phones" shows the selection and closes the menu

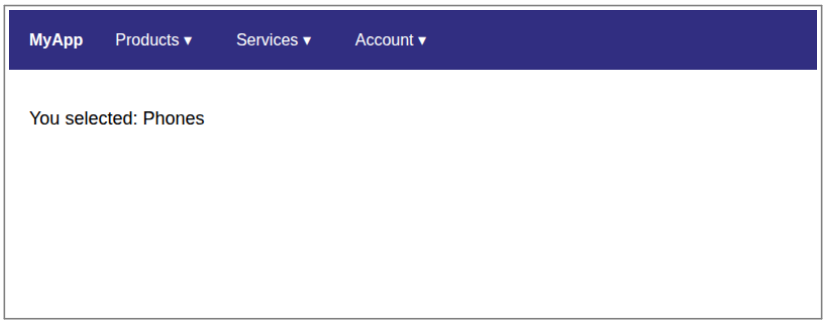


Fig 5.5 Pressing the Esc key closes any open dropdown

(d) Performance and improvements

- **Performance:** event delegation uses one click listener instead of one per link, and toggling a CSS class is cheaper than changing many style properties from JavaScript.
- **Improvements:** add arrow-key navigation inside the dropdown, a small hover delay to avoid accidental closing, a fade/slide transition, and `{passive:true}` or throttling for scroll/resize related handlers.

Conclusion: Responsive layout (Flexbox, Grid, media queries, mobile-first) and JavaScript events were combined to build adaptable, interactive web interfaces, and each design was tested at multiple screen widths.